Les Agents IA

Comprendre avant d'automatiser

Une exploration de 15 minutes pour démystifier les agents IA — du LLM aux outils, des instructions aux skills, jusqu'au protocole MCP — et comprendre comment tout cela s'assemble dans GitHub Copilot.

~15 MIN | GITHUB COPILOT | VS CODE

Made with GAMMA

L'écosystème agentic

Comment les pièces s'assemblent

Un agent IA n'est pas un simple modèle de langage. C'est un système composé de plusieurs couches qui interagissent : le contexte fourni au modèle, les capacités de l'agent lui-même, et les actions qu'il peut déclencher dans le monde réel.

Contexte

Prompt système, historique de conversation, résultats de retrieval

Agent

LLM + Instructions + Skills — le cœur du système

Actions

Tools, MCP, APIs externes, services cloud

Comprendre chaque pièce permet de mieux concevoir, configurer et superviser un agent.

Made with GAMMA

LLM, Prompt & Contexte

Le moteur et ce qu'on lui donne

⚠ Ce n'est pas de la "réflexion" — c'est de la **prédiction statistique** sur du texte et du code. Il peut produire des sorties plausibles mais fausses.

Comment fonctionne un LLM

01

Tokenisation

Le texte est découpé en **tokens** — unités de base du traitement

02

Calcul de probabilités

Le modèle calcule des probabilités pour le token suivant

03

Échantillonnage

Un mécanisme d'échantillonnage choisit la suite

04

Itération

Le processus se répète jusqu'à la fin de la réponse

Les trois couches d'un prompt

Couche	Contenu	Qui le contrôle
System	Rôle, comportement, contraintes, outils	Le produit ou le dépôt
Historique	Conversation précédente sélective	L'outil
User	La demande courante	L'utilisateur

ⓘ Deux outils peuvent produire des réponses très différentes à une demande identique si leurs system prompts divergent. Un **meilleur** contexte vaut mieux qu'un **plus gros** contexte.

Made with GAMMA

Agent

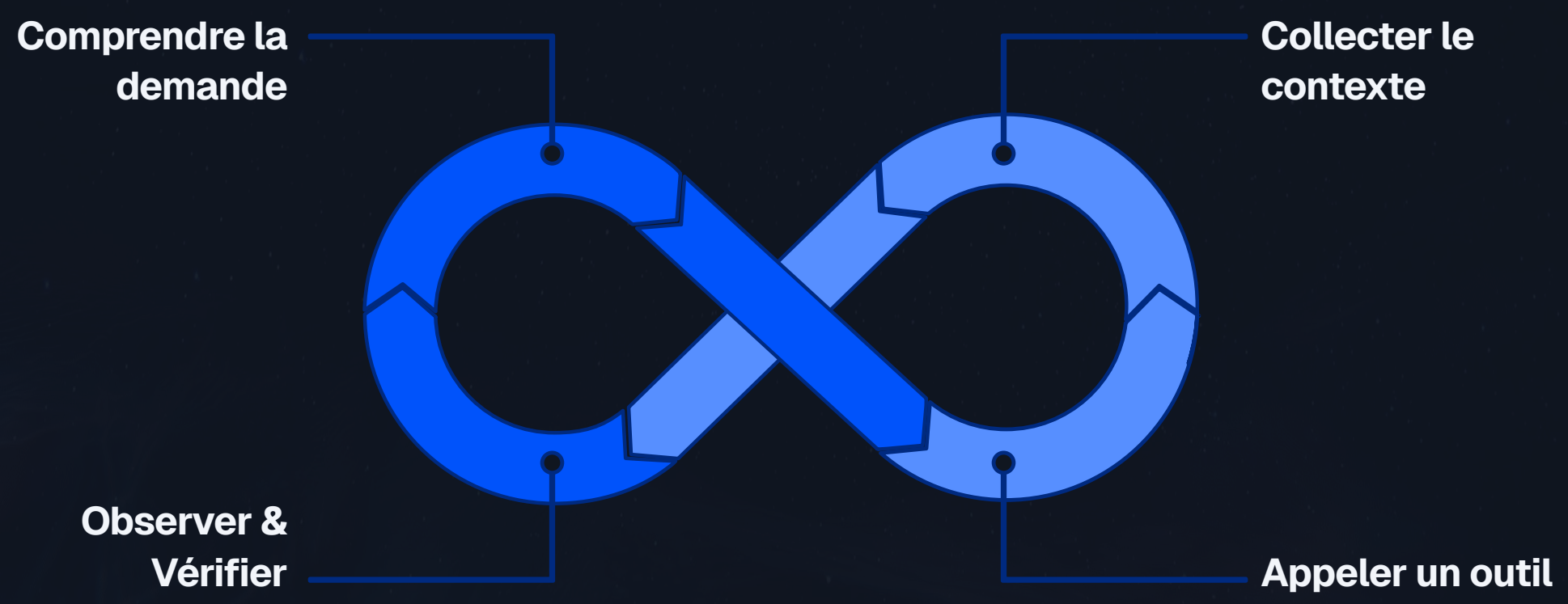
Un LLM + contexte + outils + boucle

Un chatbot répond une fois puis s'arrête. Un **agent** tourne en boucle jusqu'à l'objectif atteint — c'est le patron **ReAct** (Reasoning + Acting).

LLM vs Agent

	LLM	Agent
Ce qu'il fait	Prédit la suite de tokens	Planifie, agit, vérifie, itère
Mémoire	Contexte courant uniquement	Peut avoir une mémoire externe
Actions	Aucune par lui-même	Lit, cherche, modifie, exécute
Usage typique	Génération de texte directe	Tâches multi-étapes avec outils

Boucle agentique



Cette boucle est le cœur de tout agent : l'agent ne s'arrête pas après une seule réponse, il itère jusqu'à ce que l'objectif soit atteint ou qu'il ne puisse plus progresser.

Made with GAMMA

Tools

Les actions que l'agent peut déclencher

Un tool est une **action externe appelable par l'agent** : lire un fichier, lancer une commande, rechercher du texte, modifier un fichier, interroger une API.

Mécanisme — Function calling

1

Indication

L'agent indique l'outil à appeler et ses arguments

2

Exécution

Le **système** exécute l'outil — pas le modèle lui-même

3

Réinjection

Le résultat est réinjecté dans le contexte de l'agent

4

Continuation

L'agent continue son raisonnement avec les vraies données

Sans outils

Assistance spéculative — le LLM **devine** sans accès à l'environnement réel

Avec outils

Assistance reliée à un **environnement d'exécution réel** — lecture, écriture, tests, recherche web

- Lire et rechercher dans les fichiers du dépôt
- Exécuter des commandes dans le terminal
- Lancer les tests automatisés
- Faire une recherche web

On peut définir dans un fichier dédié quels outils sont autorisés librement, lesquels nécessitent une confirmation, et lesquels sont interdits.

Made with GAMMA

Instructions

Les règles stables qui encadrent le comportement de l'agent

Une instruction est une **règle persistante** qui oriente le comportement du système, indépendamment de la demande courante. Elle peut venir du produit, du dépôt, de l'équipe ou de l'utilisateur.

Dans GitHub Copilot → `.github/copilot-instructions.md`
Ce fichier est lu automatiquement par Copilot pour toute l'équipe.

Exemples de règles utiles

Périmètre

Préférer des patchs minimaux — ne pas refactoriser au-delà du scope demandé. Ne pas modifier les fichiers générés automatiquement.

Qualité

Vérifier les erreurs TypeScript et les tests avant de conclure. Écrire les tests en même temps que le code, pas après.

Sécurité

Ne jamais logger des données sensibles (tokens, mots de passe, données utilisateur). Valider tous les inputs avec Zod avant tout traitement.

Transparence

Résumer les risques résiduels à la fin d'une modification.

Comparaison avec les autres couches

	Instructions	Prompt	Skills
Nature	Règle stable	Demande courante	Capacité spécialisée
Fréquence	Rare	À chaque interaction	Selon le besoin
Portée	Toutes les interactions	La session courante	La tâche en cours

Made with GAMMA

Skills

Des compétences spécialisées exposées à l'agent

Une skill est une **capacité packagée** que l'agent peut charger selon le contexte : checklist de revue, processus de migration, procédure de debug…

⚠ **Les skills ne sont pas chargées automatiquement.** Elles sont découvrables à la demande, via leur champ description. Une description vague → la skill n'est jamais déclenchée.

Exemple de description efficace

```
name: security-review
description: "Use when reviewing authentication, authorization, input validation,
             or security-sensitive code. Triggers on mentions of JWT, OAuth,
             SQL queries, user inputs, or permissions."
```

Structure dans le dépôt

```
.github/skills/
  api-review/
    SKILL.md        ← description + instructions
    checklist.md    ← ressource associée
```

✓ **Ce que ça change :** plutôt que de répéter les mêmes instructions à chaque session, on écrit la skill une fois — l'agent la charge quand c'est pertinent.

Made with GAMMA

RAG

Retrieval-Augmented Generation — enrichir le contexte depuis une base

Le problème

Le LLM ne connaît pas votre codebase, vos docs internes, ou les informations récentes. Sans accès à ces sources, il génère des réponses génériques ou hallucine des détails.

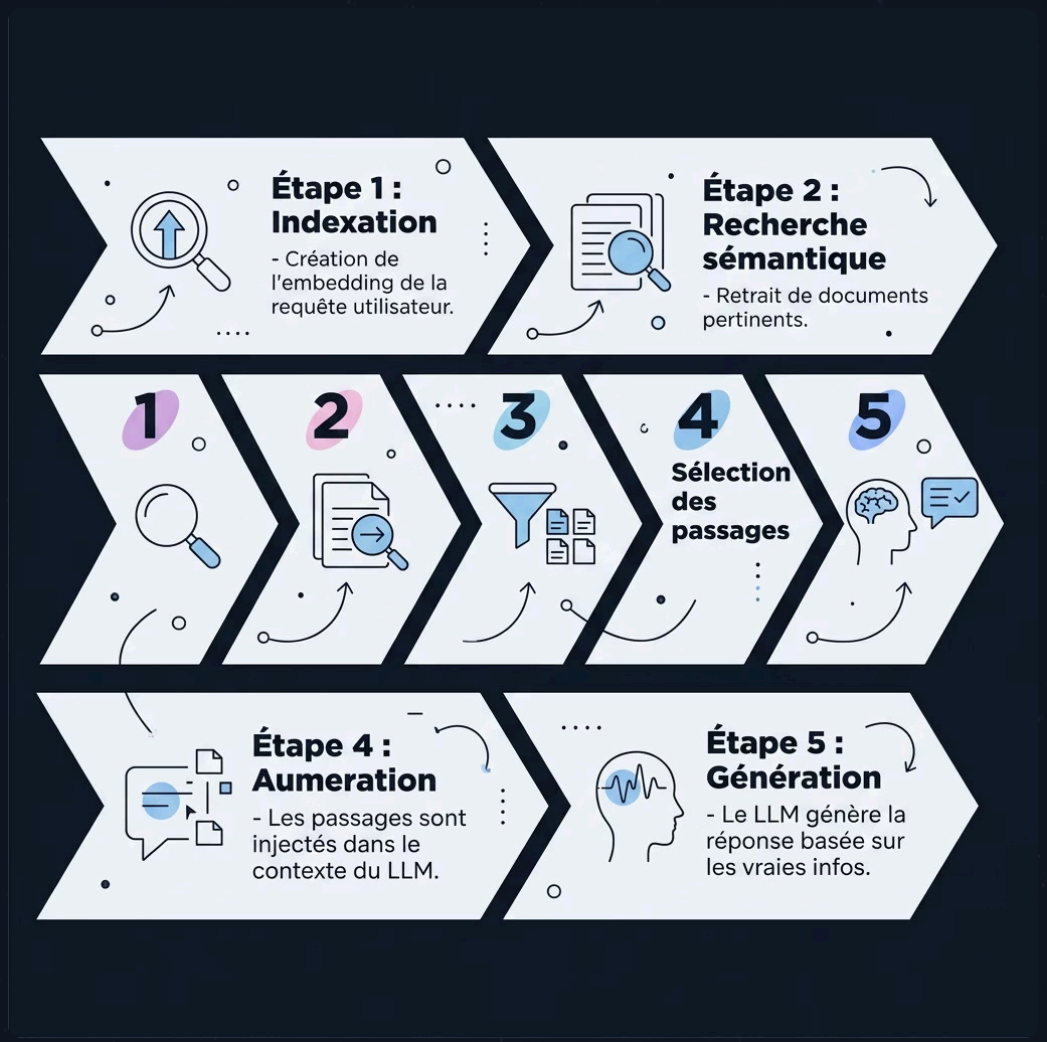
La solution — RAG

Avant de générer, le système récupère les passages pertinents dans une base documentaire et les injecte dans le contexte. La génération s'appuie alors sur de vraies informations.

Quand c'est utile

- Documentation interne volumineuse
- Base de connaissances produit
- Dépôt multi-modules avec conventions fortes

⚠ **Limites :** si la récupération est mauvaise, la génération sera mauvaise. Le RAG n'élimine pas les hallucinations — il les ancre dans de meilleures sources.



ⓘ **Et `@workspace` dans Copilot ?** `@workspace` est similaire dans son effet : Copilot indexe votre projet et récupère les passages de code pertinents avant de répondre. Techniquement, il utilise une recherche dans le code (code search) plutôt qu'une base vectorielle. Le résultat est le même : le contexte est enrichi avec les vrais fichiers du projet.

Made with GAMMA

MCP

Model Context Protocol — le port USB universel des agents

MCP est un standard ouvert (Anthropic, adopté par l'industrie) pour exposer à un agent des ressources et des outils externes de façon **uniforme**. Au lieu d'intégrer chaque source ad hoc, on écrit un serveur MCP une fois — et n'importe quel client compatible peut s'y connecter.

Le protocole

La spécification standard ouverte qui définit comment clients et serveurs communiquent

Les serveurs MCP

Exposent des outils et ressources — GitHub, Jira, bases de données, APIs externes, serveurs custom

Le client

VS Code, Copilot, Claude… qui décide comment utiliser les outils exposés

Exemples de serveurs MCP

GitHub

Issues, Pull Requests, repositories — accès direct depuis l'agent

Jira / Linear

Tickets, sprints, backlogs — l'agent peut lire et créer des tâches

Base de données

Requêtes SQL directes — l'agent interroge les vraies données

Serveur custom

En Node.js ou Python — quelques dizaines de lignes suffisent

GitHub Copilot supporte MCP → branchez vos propres outils directement dans VS Code. La bonne question : quelles ressources sont exposées, sous quelles permissions, pour quels workflows réels ?

Made with GAMMA

En pratique : Copilot + Démo

Ce qu'on peut faire maintenant dans VS Code

Les trois modes Copilot

Mode	Ce qu'il fait	Modifications	Supervision
Ask	Répond, explique, propose. Ne modifie rien.	Non	Totale
Agent	Explore le dépôt, lance des outils, enchaîne les étapes.	Oui, plusieurs fichiers	Par étape
Plan	Génère un plan que l'utilisateur valide avant exécution.	Après validation	Maximale

Plan de démo

1

`.github/copilot-instructions.md`

Créer les instructions de l'équipe, voir l'effet immédiat sur le comportement de Copilot

2

`@workspace`

"Où est géré le middleware d'auth ?" — Copilot cherche et cite les fichiers pertinents

3

Agent mode

"Ajoute une route GET /health avec son test" — observer la boucle agentique en direct

4

MCP

Montrer un serveur connecté (exemple : PokeAPI → demander les stats de Pikachu)

Un agent = LLM + contexte + outils + boucle

Pas juste un modèle — un système complet qui itère

La qualité dépend du contexte

Instructions et contexte comptent autant que le modèle lui-même

Copilot = tout l'écosystème

Retrieval (@workspace), tools, instructions, skills, MCP — tout est là

Made with GAMMA